

Failure Modes in Chained Tool-Calling Agents Operating Without Human Checkpoints

Abstract

Large language model (LLM) agents increasingly combine reasoning, planning, memory, and external tools to execute tasks that extend beyond conventional text generation. In a chained tool-calling agent, the output of one model–tool interaction becomes an input to subsequent interactions, producing a sequential execution process that may continue without human verification. This architecture creates substantial gains in task automation but also introduces distinctive reliability and safety problems: errors can propagate across calls, incorrect assumptions can become embedded in later state, tool outputs can be misinterpreted, plans can drift from the original objective, and malicious or untrusted tool content can redirect subsequent actions. These risks are amplified when actions are irreversible or when chains are long enough that small per-step error probabilities compound.

This paper develops a taxonomy of failure modes in human-checkpoint-free chained tool-calling agents. Drawing on research on tool-augmented language models, autonomous agents, interactive benchmarks, self-correction, prompt injection, and agent security, it distinguishes six principal classes of failure: **planning and decomposition failures; tool-selection and argument failures; state, memory, and context failures; execution and environment failures; verification and recovery failures; and security and governance failures**. The paper argues that the central problem is not simply whether an LLM can make an individual correct tool call, but whether the *entire trajectory* remains correct, safe, and policy-compliant as intermediate outputs become inputs to future actions.

Current approaches—including ReAct-style interleaving, tool-use fine-tuning, reflective methods such as Reflexion and Self-Refine, benchmark-based evaluation,

prompt-injection defenses, prospective tool-safety assessment, and emerging formal safety mechanisms—mitigate particular failure modes but do not yet provide reliable guarantees for long-horizon autonomous execution. The paper identifies research gaps in trajectory-level reliability metrics, causal attribution of failures, calibrated uncertainty, reversible execution, adaptive checkpointing, formal verification, and evaluation under realistic distribution shift. It concludes that safe autonomy should not be conceptualized as the elimination of human checkpoints, but as their principled replacement with machine-verifiable control boundaries, risk-sensitive intervention policies, and independently validated execution constraints.

Keywords: LLM agents; tool calling; autonomous agents; chained tool use; agent reliability; error propagation; prompt injection; agent safety; long-horizon reasoning; human-in-the-loop systems

1. Introduction

Large language models have evolved from passive text-generation systems into components capable of interacting with external environments through APIs, browsers, databases, code interpreters, search systems, and other computational tools. Toolformer demonstrated that language models can learn when to invoke APIs, which arguments to provide, and how to incorporate returned information into subsequent generation (Schick et al., 2023). ReAct subsequently established a widely influential pattern in which reasoning and acting are interleaved, allowing an agent to alternate between internal deliberation and environmental feedback (Yao et al., 2023). More recent systems and benchmarks have expanded this paradigm toward long-horizon web interaction, software engineering, embodied environments, and real-world API workflows.

The resulting systems differ fundamentally from single-turn LLM applications. A conventional chatbot may produce an incorrect answer that a human can inspect before acting on it. A chained agent can instead generate an action, execute it, consume its output, generate another action, execute that action, and continue until a task is apparently complete. Consequently, the system's intermediate representations are not merely explanatory text: they become operational state. A mistaken assumption at step (i) can therefore alter the input distribution encountered at step (i+1), creating opportunities for *error propagation* and *trajectory divergence*.

This distinction becomes particularly important when no human checkpoint exists between consequential actions. A human checkpoint can function as an error-correction boundary: a person can notice that an intermediate result is inconsistent with the goal, reject an unsafe action, correct an incorrect assumption, or terminate execution. Removing that boundary transfers responsibility for detection, interpretation, and recovery to the agent architecture itself. Research on

interactive benchmarks already demonstrates that high capability on individual language tasks does not translate into reliable autonomous execution. AgentBench identifies long-term reasoning, decision-making, and instruction-following as major obstacles; WebArena reports a large gap between human and autonomous-agent performance on realistic, long-horizon web tasks; and τ -bench finds that even advanced function-calling agents can be inconsistent across repeated trials.

The central thesis of this paper is therefore:

The reliability of a chained tool-calling agent is a trajectory-level property, not merely a property of the individual model, tool, or call.

This thesis motivates a failure-oriented analysis. Rather than asking whether an agent *can* use tools, the relevant questions are: What can go wrong during a chain? How does an early failure affect later actions? Which failures are detectable? Which are recoverable? Which become irreversible before detection? And what architectural mechanisms can replace human checkpoints without assuming that the same LLM can perfectly verify its own decisions?

2. Background and Related Work

2.1 From language models to tool-using agents

Tool augmentation addresses a fundamental limitation of language models: their internal parameters do not provide guaranteed access to current databases, exact arithmetic, external state, or executable environments. Toolformer showed that an LLM can be trained to determine whether and how to invoke external APIs (Schick et al., 2023). ToolLLM extended this direction to thousands of real-world APIs and explicitly considered multi-tool solution paths, demonstrating the increasing complexity of tool-use planning (Qin et al., 2024).

ReAct framed tool use as an iterative reasoning–action loop. Rather than producing a complete plan and executing it blindly, an agent can reason, act, observe the environment, and revise its subsequent behavior (Yao et al., 2023). This is important for robustness because observations can correct mistaken assumptions. Nevertheless, the same loop introduces another dependency: the model must correctly interpret every observation on which later decisions depend.

Contemporary surveys characterize LLM agents around several interacting capabilities, commonly including reasoning, planning, memory, perception, tool use, and action. Recent reviews emphasize that reliability, alignment, generalization, and evaluation remain unresolved problems as agents move toward more autonomous operation (Wang et al., 2024; Chowa et al., 2026).

2.2 Long-horizon autonomy

Long-horizon tasks expose weaknesses that are less visible in single-step benchmarks. WebArena, for example, constructs realistic web environments with functioning websites and long-horizon tasks. Its initial experiments showed only 14.41% end-to-end success for the strongest GPT-4-based agent compared with 78.24% for humans (Zhou et al., 2024).

AgentBench similarly evaluates agents across eight environments and reports that poor long-term reasoning, decision-making, and instruction following are significant causes of failure (Liu et al., 2024). Mind2Web demonstrates that realistic websites introduce substantial diversity in domains, interfaces, and interaction patterns, making generalization considerably harder than operating in simplified environments (Deng et al., 2023).

These findings suggest that autonomy introduces a *depth dimension* to reliability. If an agent must execute (n) dependent actions, the relevant probability is not simply the accuracy of a single call. Under the simplifying assumption of independent step correctness (p), the probability that every step is correct is:

$$P(\text{all correct}) = p^n.$$

Thus, even ($p=0.99$) yields approximately (0.90) probability of an entirely correct 10-step chain and approximately (0.61) for a 50-step chain. Real systems are not independent, so this expression is not a predictive model; rather, it illustrates why high single-step accuracy can coexist with poor trajectory reliability. Correlated errors, adaptive recovery, retries, and heterogeneous tool risks can make actual behavior substantially better or worse.

2.3 Self-correction and reflection

Several approaches attempt to reduce error accumulation without requiring human intervention. Self-Refine uses iterative feedback and revision, while Reflexion stores verbal reflections derived from environmental feedback for use in subsequent attempts (Madaan et al., 2023; Shinn et al., 2023). Voyager similarly incorporates execution errors, environment feedback, and self-verification into an iterative embodied-agent architecture (Wang et al., 2023).

These approaches establish an important principle: autonomy need not mean one-shot execution. An agent can introduce internal verification loops. However, self-correction also raises a fundamental *verification-ceiling* problem. If the same model generates an action and judges that action using similar information and inductive biases, the verifier may reproduce the generator's mistake. Moreover, reflection is useful only when the

system receives a sufficiently informative error signal. A semantically plausible but incorrect database result, for example, may provide no explicit signal that a failure occurred.

3. Failure Taxonomy for Chained Tool-Calling Agents

A chained tool-calling agent can be represented as a partially observed sequential decision process:

$$[\begin{array}{l} s_t \xrightarrow{\pi_\theta} a_t \\ \xrightarrow{T} o_{t+1} \\ \xrightarrow{\pi_\theta} a_{t+1}, \end{array}]$$

where (s_t) is the agent's internal state, (a_t) is a tool action, (T_t) is the external tool/environment transition, and (o_{t+1}) is the resulting observation. The next decision depends not only on the original objective but also on accumulated observations and memory.

This creates six interconnected failure classes.

3.1 Planning and task-decomposition failures

The first class concerns the construction and maintenance of the action sequence.

Goal misinterpretation. Natural-language instructions are frequently underspecified. An agent may infer a plausible but incorrect interpretation and construct a coherent chain around it. Because later calls are conditioned on the initial interpretation, the error can persist without becoming syntactically visible.

Incorrect decomposition. A complex task may require dependencies that the agent fails to represent. For example, an agent asked to identify, validate, and update records might execute the update before completing validation.

Premature commitment. Agents can select a particular plan before acquiring information that would distinguish alternative strategies. ReAct mitigates this to some extent by interleaving observation and action, but the agent still determines when to gather information and when to commit to an action (Yao et al., 2023).

Plan drift. Long chains may gradually optimize for local subgoals rather than the original objective. Each individual action can appear reasonable while the overall trajectory becomes inconsistent with the user's intended outcome. AgentBench's findings on long-term reasoning and instruction-following

limitations are consistent with these failure modes (Liu et al., 2024).

3.2 Tool-selection and argument failures

Even when the high-level plan is correct, the agent must select the correct tool and construct valid parameters.

Wrong-tool invocation occurs when several tools offer superficially similar functionality. A search API, database query, and knowledge-retrieval function may all appear capable of answering a request, while differing substantially in freshness, authority, side effects, or access permissions.

Argument hallucination occurs when the model invents unsupported fields, identifiers, parameter values, or assumptions about tool semantics. ToolLLM demonstrates the scale of the problem by explicitly training and evaluating models over thousands of APIs, highlighting the difficulty of generalizing tool-use behavior across heterogeneous interfaces (Qin et al., 2024).

Schema misunderstanding is particularly dangerous when an API accepts syntactically valid but semantically incorrect values. A tool call can therefore be *well-formed yet wrong*.

Over-privileged tool choice is another failure mode. An agent may use a high-impact mutation tool when a lower-risk read-only tool would have been sufficient. This converts a reasoning error into an operational risk.

3.3 State, memory, and context failures

Chained agents depend on maintaining an accurate representation of what has already happened.

A first problem is **state desynchronization**: the agent believes that an action succeeded even though the environment rejected it, partially executed it, or changed the result asynchronously.

A second is **context degradation**. As trajectories grow, earlier instructions and observations may become difficult to retrieve or correctly prioritize. Research on long-context language models has shown that performance can degrade when relevant information appears in the middle of long contexts, even when the model nominally supports large context windows (Liu et al., 2024).

A third is **memory contamination**. Persistent memory can preserve erroneous assumptions and make future actions systematically biased. Agent Security Bench identifies vulnerabilities associated not only with prompts and tool use but also with memory retrieval, demonstrating that memory constitutes an attack and failure surface rather than an

unqualified reliability mechanism (Zhang et al., 2025).

The resulting phenomenon may be called **stateful error persistence**: an error stops being a local mistake and becomes part of the agent's working model of the world.

3.4 Tool and environment failures

Not all failures originate in the model. Tools themselves can fail through timeouts, stale data, authentication changes, rate limits, malformed responses, API version changes, partial execution, or inconsistent external state.

A robust agent must distinguish:

1. **No result** — the tool failed to return an answer.
2. **Invalid result** — the tool returned an answer that violates schema or consistency constraints.
3. **Stale result** — the result is syntactically valid but no longer represents current state.
4. **Adversarial result** — the result contains instructions designed to manipulate the agent.
5. **Ambiguous result** — multiple interpretations remain possible.

Conflating these cases is itself a failure mode. A timeout should not necessarily trigger the same recovery policy as a successful but suspicious database mutation.

3.5 Verification and recovery failures

A chain without human checkpoints requires the system to decide when it should trust itself.

This creates a distinction between **generation confidence** and **execution correctness**. A model may be highly confident while being wrong. Moreover, chain-of-thought-like reasoning should not automatically be interpreted as a faithful explanation of the causal basis for model behavior; research has demonstrated systematic cases in which generated reasoning does not faithfully reflect the mechanisms underlying answers (Turpin et al., 2023).

Self-Refine and Reflexion demonstrate that iterative feedback can improve performance, but their effectiveness depends on feedback quality (Madaan et al., 2023; Shinn et al., 2023).

This creates several failure modes:

- **False verification**: the verifier approves an incorrect action.
- **Verification blindness**: the system checks syntax but not semantics.
- **Late verification**: an error is detected only after irreversible actions.
- **Error masking**: retries produce a superficially successful state while

obscuring the original failure.

- **Recovery divergence:** an attempted recovery introduces a second independent error.

The absence of a human checkpoint therefore does not merely remove a safety layer; it requires the system to implement *machine-mediated epistemic judgment* about whether its own trajectory remains trustworthy.

3.6 Security and governance failures

The final category concerns deliberate manipulation and policy violations.

Prompt injection is especially important because tool outputs can contain untrusted natural-language content. AgentDojo explicitly evaluates agents operating on untrusted data and shows that external information can hijack an agent's subsequent tool use (Debenedetti et al., 2024). Its benchmark includes realistic tasks involving email, banking, and travel, illustrating that the attack surface emerges precisely because agents transform information into actions.

Agent Security Bench further demonstrates vulnerabilities across system prompts, user prompts, tool usage, and memory retrieval, while AgentHarm shows that agents can exhibit harmful multi-step behavior when their safeguards are bypassed or insufficiently robust (Zhang et al., 2025; Andriushchenko et al., 2025).

The security problem is consequently not reducible to "prompt safety." A tool-enabled agent can become a *confused deputy*: the model possesses legitimate authority but is induced by untrusted content to apply that authority toward an unintended objective.

4. Error Propagation and Cascading Failure

The distinctive characteristic of chained tool calling is that errors can become *inputs* to future decisions.

Consider a sequence:

[
a₁ $\rightarrow$ o₂ $\rightarrow$ a₂ $\rightarrow$ o₃
 $\rightarrow$ a₃ $\rightarrow$ $\cdots$ $\rightarrow$ a_n.
]

Suppose (o₂) contains a subtle error. The model generating (a₂) does not necessarily know that (o₂) is erroneous. Consequently, (a₂) may be internally rational conditional on false information. Its output can then generate (o₃), which further reinforces the mistaken trajectory.

This produces four patterns of cascading failure.

Linear propagation occurs when one error directly causes the next incorrect action.

Branching propagation occurs when one erroneous assumption influences multiple subsequent calls.

Feedback amplification occurs when later tool results appear to confirm an incorrect assumption, strengthening the model's confidence.

Irreversible propagation occurs when the chain performs an action that cannot easily be undone, such as deleting records, sending external communications, transferring resources, or modifying production systems.

A useful risk model therefore needs to distinguish *step accuracy* from *trajectory integrity*. Let (E_t) denote whether the agent remains within the acceptable state set after step (t) . Then:

$$\begin{aligned} &[\\ &P(E_{\{1:n\}}) = \\ &P(E_1) \prod_{t=2}^n P(E_t \mid E_{\{1:t-1\}}). \\ &] \end{aligned}$$

This formulation emphasizes that later reliability is conditional on the entire prior trajectory. It also shows why benchmarking individual function-call accuracy is insufficient.

The τ -bench results are especially relevant. The benchmark evaluates dynamic user–agent conversations in domains with APIs and policy rules and introduces (pass^k) to capture repeated-trial reliability. Its reported results show both sub-50% task success for leading function-calling systems and substantial inconsistency across trials (Yao et al., 2025).

5. Current Approaches and Developments

5.1 Structured tool-use training

Toolformer and ToolLLM represent two influential directions: learning tool invocation from demonstrations and constructing large-scale tool-use datasets. These methods improve tool selection and argument generation, but they do not by themselves solve trajectory-level safety. Tool competence is necessary for autonomy but not sufficient for trustworthy autonomy.

5.2 Reasoning–acting loops

ReAct's central contribution is to replace purely open-loop planning with an observe–reason–act cycle. This can reduce hallucination by allowing the model to query the environment rather than relying exclusively on parametric knowledge (Yao et al., 2023).

However, ReAct remains vulnerable if the observations themselves are unreliable, adversarial, or misinterpreted. Thus, feedback does not automatically equal verification.

5.3 Reflection and self-correction

Reflexion and Self-Refine introduce internal feedback loops that allow agents to revise behavior without weight updates or human intervention. These approaches are attractive because they convert failed attempts into additional information and can improve performance at inference time (Shinn et al., 2023; Madaan et al., 2023).

The principal limitation is that a model cannot necessarily detect errors for which it lacks an independent source of truth. Reflection is strongest when external feedback is objective—for example, compiler success or a test-suite result—and weaker when the correctness criterion is ambiguous or itself linguistic.

5.4 Benchmark-driven evaluation

The research community has increasingly moved toward interactive benchmarks rather than static question answering. AgentBench evaluates multiple environments; WebArena emphasizes realistic web tasks; τ -bench tests policy compliance and repeated interaction; AgentDojo focuses on security under untrusted data; and Agent-SafetyBench evaluates multiple classes of agent safety risk.

This development is important because autonomous systems need evaluation at the level where failures actually occur: trajectories, state transitions, tool decisions, and consequential outcomes.

5.5 Prospective safety mechanisms

An important recent development is the shift from evaluating whether an agent *already caused harm* toward evaluating whether an intended tool invocation is unsafe *before execution*. SafeToolBench explicitly adopts this prospective perspective, arguing that retrospective evaluation can be inadequate when tool actions are irreversible (Xia et al., 2025).

Similarly, recent work on verifiably safe tool use proposes translating hazards into explicit constraints on tool sequences and data flows, including capability, confidentiality, and trust labels. The approach is notable because it seeks guarantees at the system level rather than relying solely on the LLM's judgment

(Doshi et al., 2026).

This direction is particularly promising for replacing some human checkpoints: instead of asking the model whether an action is safe, the infrastructure can enforce constraints independently of the model.

6. Research Challenges and Limitations

6.1 Defining reliability

Agent reliability is multidimensional. At minimum, evaluation should distinguish:

- **task success** — whether the final objective was achieved;
- **trajectory correctness** — whether intermediate actions were appropriate;
- **policy compliance** — whether all actions obeyed constraints;
- **state integrity** — whether external state remained consistent;
- **security** — whether the agent resisted adversarial manipulation;
- **recoverability** — whether failures could be detected and reversed;
- **consistency** — whether repeated executions produce dependable outcomes.

A final-answer metric cannot distinguish an agent that succeeds through a safe trajectory from one that succeeds accidentally after several dangerous actions.

6.2 Attribution of failure

A failed trajectory may involve the model, tool, environment, memory subsystem, retrieval layer, interface, or evaluator. Current benchmarks often report whether the task succeeded without providing sufficient causal attribution.

Future systems require *failure provenance*: a structured record identifying the earliest divergence from the intended trajectory, the uncertainty associated with that decision, and the downstream consequences.

6.3 Benchmark realism

Benchmarks remain constrained representations of deployment. Real environments contain changing APIs, authentication failures, inconsistent data, ambiguous goals, multiple users, organizational policies, and adversarial inputs.

Agent-SafetyBench and ASB broaden evaluation considerably, but their environments still cannot represent the full diversity of production systems (Zhang et al., 2025).

6.4 Self-verification limitations

Self-correction is attractive because it is inexpensive relative to human review. Yet the same model may serve as planner, executor, critic, and verifier. This creates correlated failure risk.

Independent verification can be implemented through different models, deterministic validators, typed schemas, simulators, test suites, policy engines, or cryptographic authorization mechanisms. The research challenge is determining when independence materially improves reliability and when it merely increases computational cost.

6.5 Irreversibility and asymmetric risk

Not all actions should receive identical autonomy. A mistaken informational query may have negligible consequences; a mistaken deletion may be catastrophic.

This implies that agent autonomy should be **risk-sensitive** rather than binary. A system should require stronger verification as potential impact, irreversibility, uncertainty, or privilege increases.

7. Research Gaps and Future Directions

7.1 Adaptive checkpointing rather than checkpoint elimination

The most important conceptual gap is the assumption that successful autonomy means removing human checkpoints entirely. A more defensible objective is **adaptive checkpoint placement**.

The agent could dynamically estimate:

$$\begin{aligned} &[\\ &R_t = f(U_t, I_t, P_t, C_t), \\ &] \end{aligned}$$

where (U_t) is uncertainty, (I_t) is action impact, (P_t) is privilege, and (C_t) is estimated downstream consequence. A checkpoint or stronger verification mechanism would be triggered when (R_t) exceeds a threshold.

This converts human oversight from a fixed workflow step into a risk-sensitive

control policy. **7.2 Trajectory-level reliability metrics**

Future benchmarks should report metrics such as:

- probability of completing the entire chain correctly;
- probability of violating a policy at least once;
- earliest-failure position;
- expected number of unsafe actions before detection;
- recovery success rate;
- irreversible-action error rate;
- cross-trial consistency;
- calibration of uncertainty before consequential actions.

The (pass^k) idea in τ -bench provides an important starting point because reliability across repeated trials is more informative for autonomous systems than a single pass rate (Yao et al., 2025).

7.3 Typed and verifiable tool interfaces

Natural-language tool descriptions should increasingly be supplemented with machine-checkable specifications:

- input and output types;
- preconditions;
- postconditions;
- side-effect classifications;
- authorization requirements;
- data sensitivity labels;
- reversibility;
- transaction boundaries.

The emerging work on formalizing safe tool use illustrates how such specifications could move agent safety toward enforceable constraints rather than purely prompt-based behavioral expectations (Doshi et al., 2026).

7.4 Transactional and reversible autonomy

Borrowing from database and distributed-systems principles, agents could execute high-impact chains transactionally:

1. plan;
2. simulate or stage actions;
3. validate predicted state;
4. execute;
5. verify postconditions;
6. commit or roll back.

This architecture would reduce the cost of mistakes by making many agent actions reversible. Research should investigate whether tool ecosystems can

expose standardized transactional semantics suitable for agent execution.

7.5 Independent verification architectures

A promising direction is separation of roles:

- **planner** proposes the trajectory;
- **executor** performs authorized actions;
- **verifier** evaluates preconditions and postconditions;
- **policy engine** enforces non-negotiable constraints;
- **monitor** detects anomalous trajectories.

This is preferable to asking a single LLM to perform all functions. It also enables heterogeneous verification: deterministic validators can handle schemas, specialized models can evaluate domain semantics, and formal policies can constrain privileged actions.

7.6 Security-aware memory

Persistent memory should be treated as a security boundary. Memory entries require provenance, confidence, timestamps, access control, and possibly expiration. Agents should distinguish *observed facts*, *model-generated hypotheses*, *user assertions*, and *untrusted external content*.

This is particularly important given evidence that memory retrieval can become an attack surface in agent systems (Zhang et al., 2025).

7.7 Formal methods for agent workflows

The long-term research goal should be to establish properties such as:

If a tool chain begins in a state satisfying precondition (P), and all authorized tools satisfy their specifications, then execution cannot reach a state violating safety property (S).

LLMs may remain responsible for high-level interpretation and planning, while formal methods constrain what the generated plan is permitted to execute. The recent movement toward formally specified tool safety suggests a viable bridge between probabilistic language reasoning and deterministic systems engineering (Doshi et al., 2026).

7.8 Evaluation under adversarial and distribution-shift

conditions An agent that performs reliably on known benchmarks

may fail when:

- tool schemas change;
- data sources become adversarial;
- users provide contradictory instructions;
- context length increases;
- external state changes between calls;
- APIs return unexpected formats;
- the task contains hidden constraints.

Evaluation should therefore deliberately perturb tools, observations, memory, and environment state. AgentDojo and ASB provide useful foundations for this security-oriented evaluation philosophy (Debenedetti et al., 2024; Zhang et al., 2025).

8. Conclusion

Chained tool-calling agents represent a major transition from language generation toward autonomous computation. Their value arises precisely because they can transform a high-level instruction into a sequence of externally consequential actions. Yet the same property creates a distinctive class of reliability and safety problems that cannot be adequately characterized by single-step tool-call accuracy.

The central failure mechanism is **dependency**. In a chained trajectory, the output of one action becomes the context for the next. Consequently, incorrect planning, hallucinated arguments, misinterpreted tool results, corrupted memory, prompt injection, or environmental anomalies can propagate through subsequent decisions. Long-horizon benchmarks such as AgentBench, WebArena, and τ -bench demonstrate that this problem is already observable in empirical evaluations, while AgentDojo, AgentHarm, Agent-SafetyBench, and ASB show that security risks become more consequential when agents can act through tools.

Current techniques—ReAct, tool-use training, reflective reasoning, self-correction, safety prompting, and benchmark-driven evaluation—provide meaningful progress but do not establish reliable autonomy. In particular, self-correction cannot be assumed to solve failures that the system cannot independently observe, and language-based reasoning cannot substitute for authoritative verification of consequential state.

The appropriate research objective is therefore not simply **human-free execution**. It is **verifiable autonomy**: systems capable of operating independently while recognizing uncertainty, respecting explicit constraints, maintaining state integrity, detecting trajectory divergence, and escalating or stopping when the consequences exceed their verified competence.

The most promising future architecture is consequently a layered one in which probabilistic LLM reasoning is surrounded by deterministic or independently validated controls: typed tools, authorization boundaries, policy engines, transaction semantics, postcondition checks, provenance-aware memory, adaptive checkpoints,

and formal safety specifications. Such mechanisms can transform human checkpoints from a universal requirement into one component of a broader, risk-sensitive control architecture.

In this sense, the defining challenge of chained tool-calling agents is not whether models can act autonomously. It is whether autonomous action can remain **correct, secure, recoverable, and accountable over an entire trajectory**. Solving that problem is likely to be a prerequisite for deploying long-horizon agents in settings where failure is more costly than non-completion.